

Akash - Infrastructure / SRE Intern Assignment

GitHub Link: <https://github.com//zeotap-ims-assignment>

Project Summary

This repository implements a resilient Incident Management System for high-volume distributed-stack signals. It includes an async FastAPI backend, a React dashboard, debounced incident creation, mandatory RCA closure, MTTR calculation, rate limiting, health checks, throughput logging, sample data, and tests.

Architecture

Signal producers send JSON payloads to the FastAPI ingestion API. A sliding-window rate limiter protects the edge. Accepted signals enter a bounded asyncio queue so persistence slowness does not crash ingestion. Async workers write raw signals to an append-only JSONL data lake, structured incidents/RCA to SQLite, per-minute aggregates to SQLite, and active state to an in-memory dashboard cache. The React dashboard reads incidents, raw signals, status, and RCA state from the backend.

Key Requirements Covered

Async processing, 10-second component debouncing, separate raw/structured/cache/aggregate storage paths, Strategy pattern alert routing, state-machine workflow transitions, mandatory RCA before CLOSED, MTTR calculation, rate limiting, /health endpoint, throughput metrics every 5 seconds, Docker Compose packaging, sample failure data, and RCA validation tests.

Run Instructions

```
docker compose up --build
```

Dashboard: <http://localhost:5173>

Backend health: <http://localhost:8000/health>

API docs: <http://localhost:8000/docs>

Seed sample data: `python scripts/seed_failure.py`

Tests

```
cd backend && pip install -r requirements.txt && pytest
```

Backpressure

The API returns after enqueueing signals into a fixed-size async queue. When the queue is full, the response reports rejected items instead of allowing memory growth or process failure. Worker persistence uses retry with exponential backoff.

Bonus / Non-Functional Work

Input validation with Pydantic, restricted CORS configuration, Docker health checks, explicit rate limiting, transactional RCA/status updates, async-safe write locking, retry logic, and documentation for architecture, prompts, plans, setup, and backpressure.